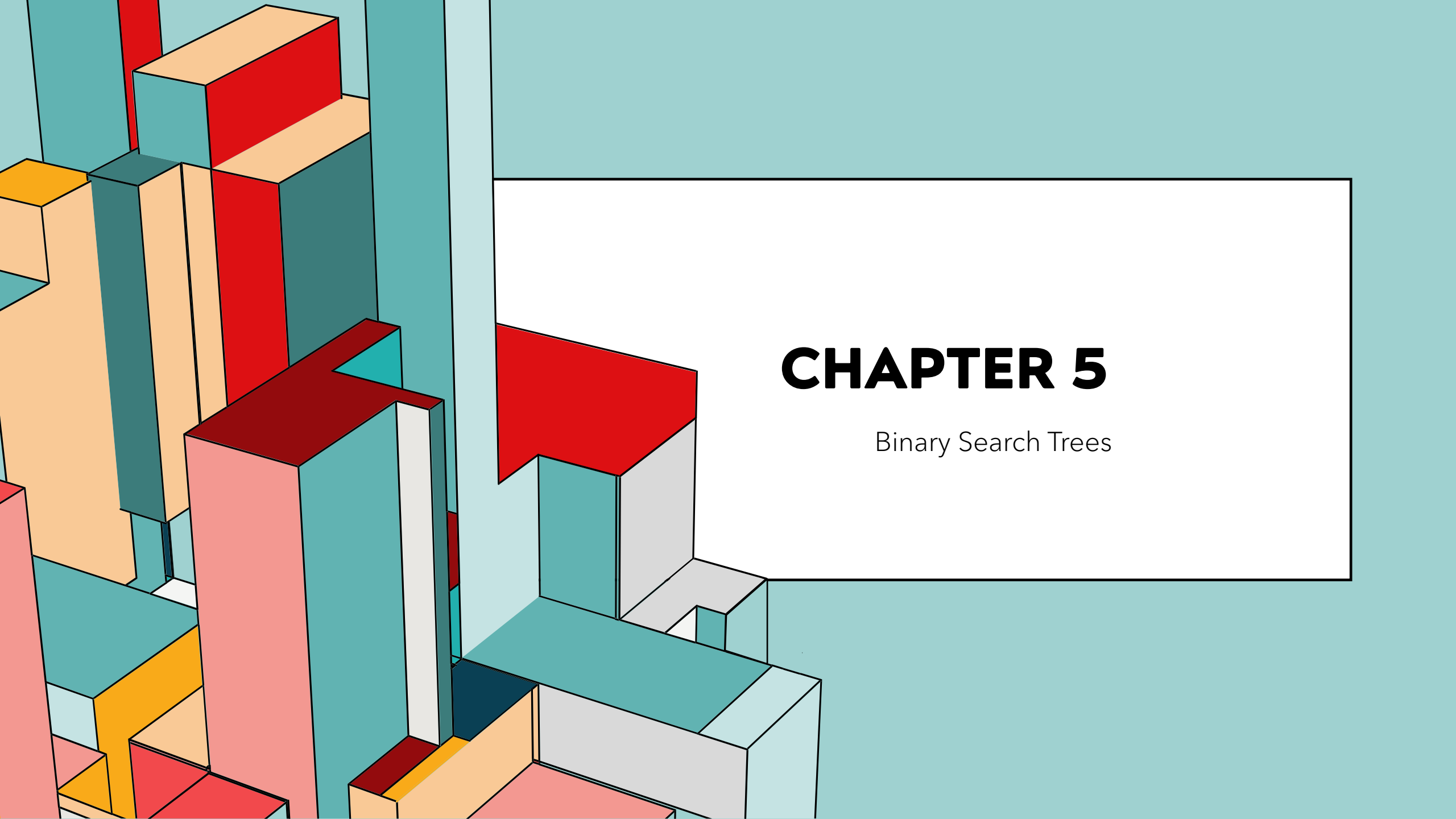


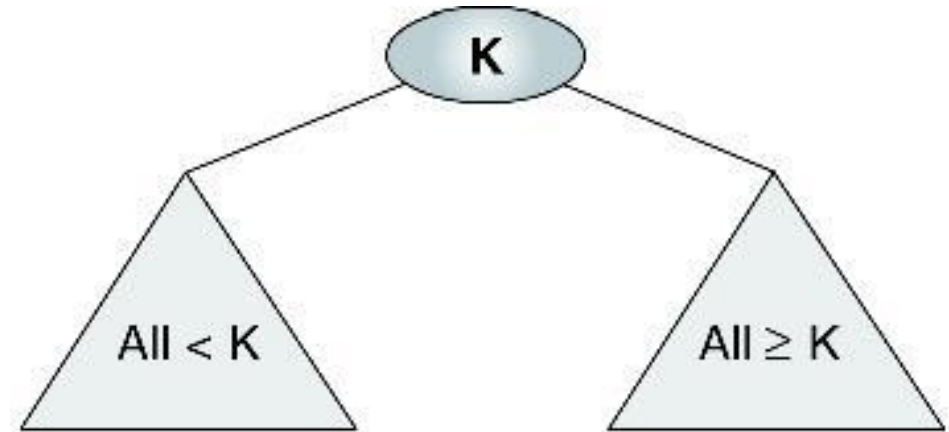


CHAPTER 5

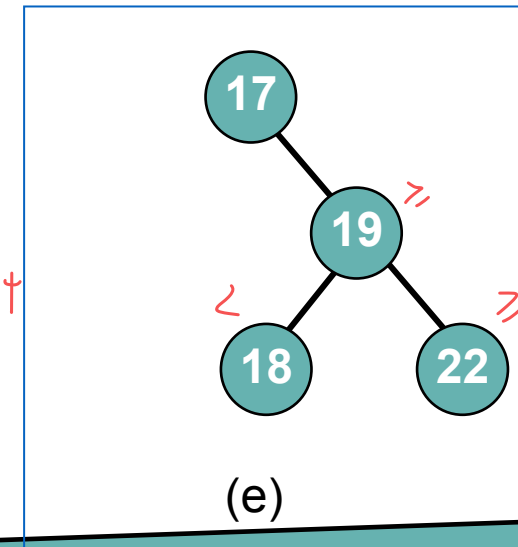
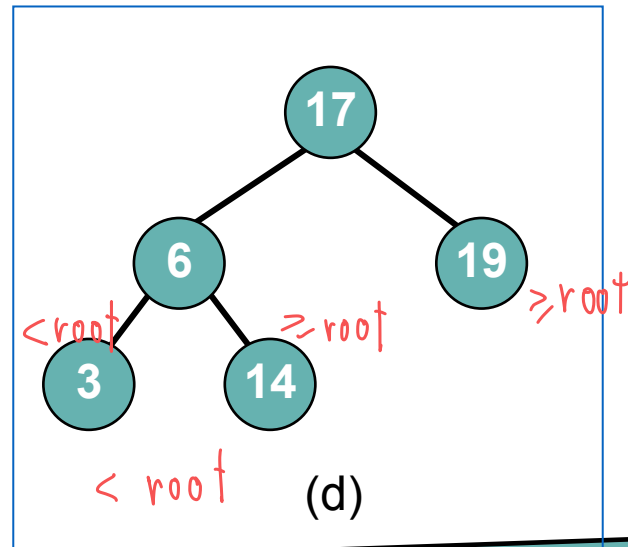
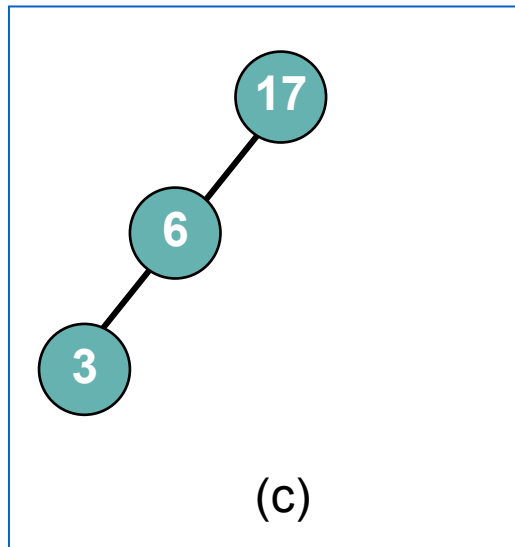
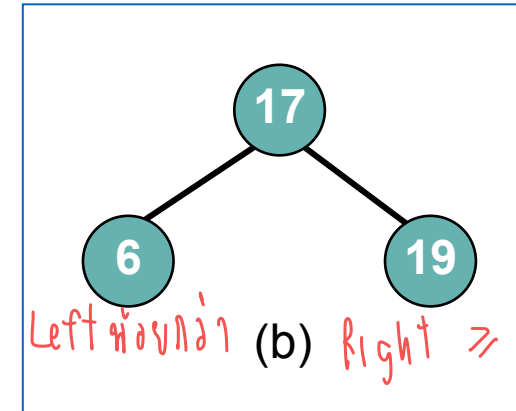
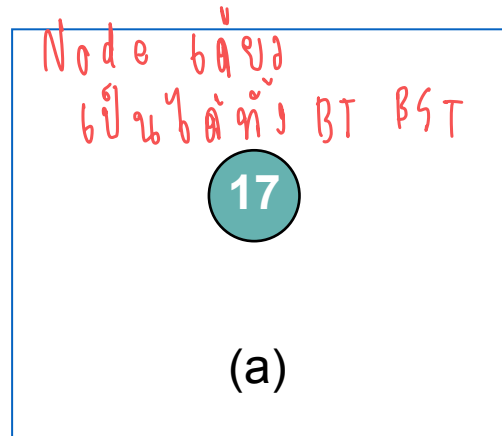
Binary Search Trees

BINARY SEARCH TREE

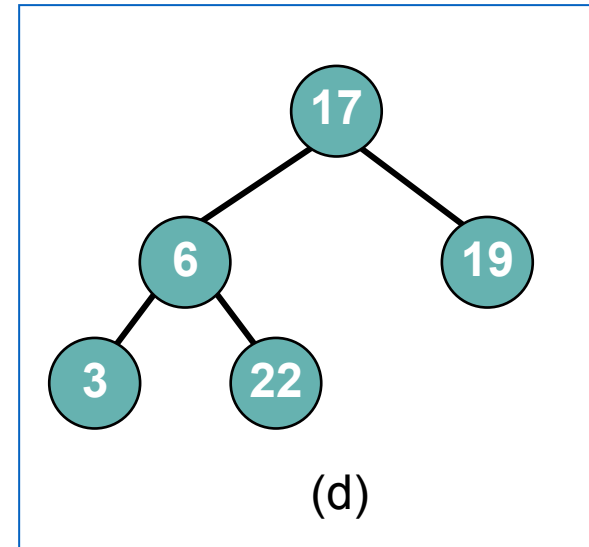
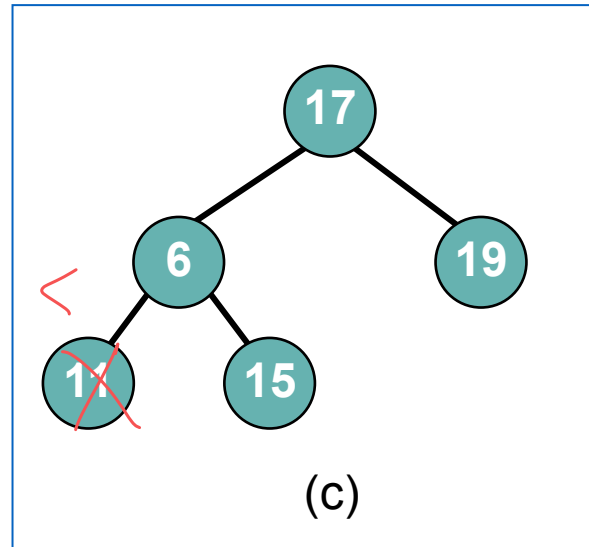
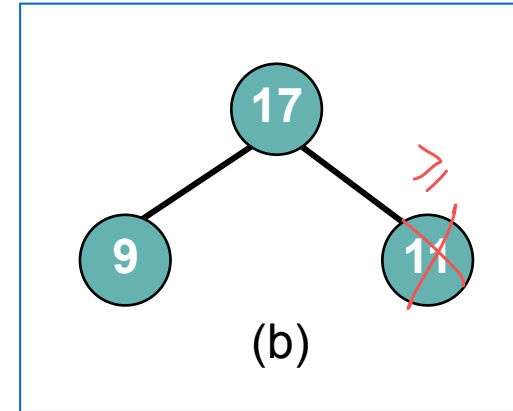
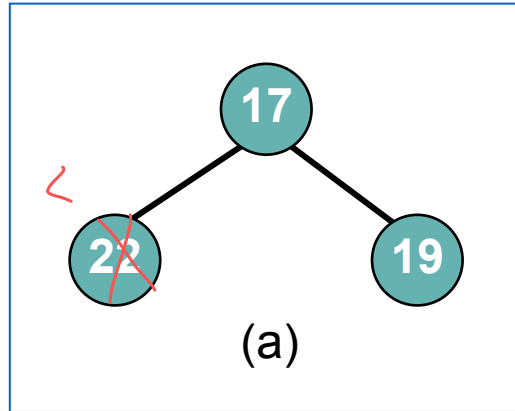
- ❖ เป็น Binary Tree ประเภทหนึ่ง โดยกำหนดให้
 - ทุกโหนดใน left subtree ต้องมีค่าน้อยกว่า root
 - ทุกโหนดใน Right Subtree ต้องมีค่ามากกว่าหรือเท่ากับ root
 - ในแต่ละ Subtree ต้องคงคุณสมบัติของ Binary Search Tree



VALID BINARY SEARCH TREES



INVALID BINARY SEARCH TREES



BST REPRESENTATIONS



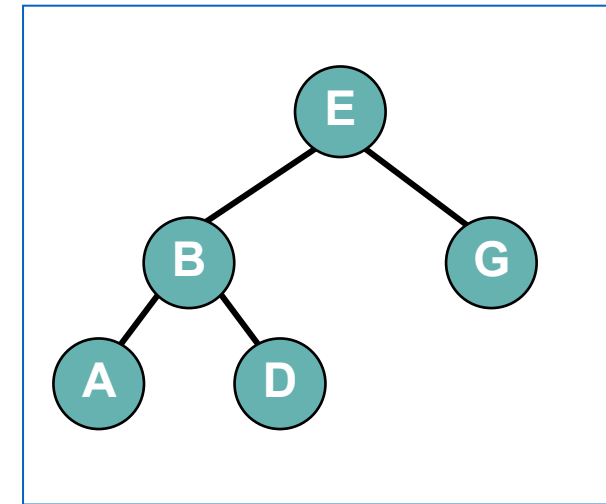
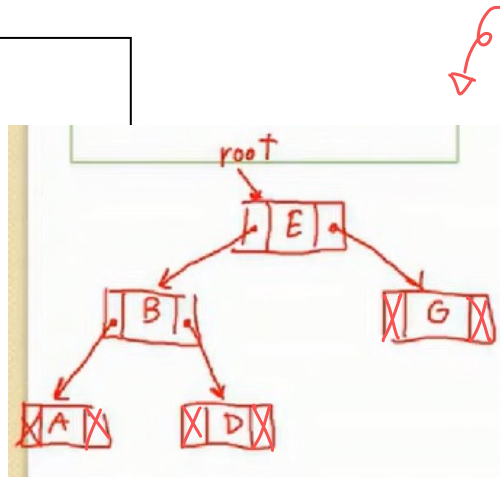
BSTNode

dataType data

BSTNode left, right

end BSTNode

BSTNode root



BST OPERATIONS

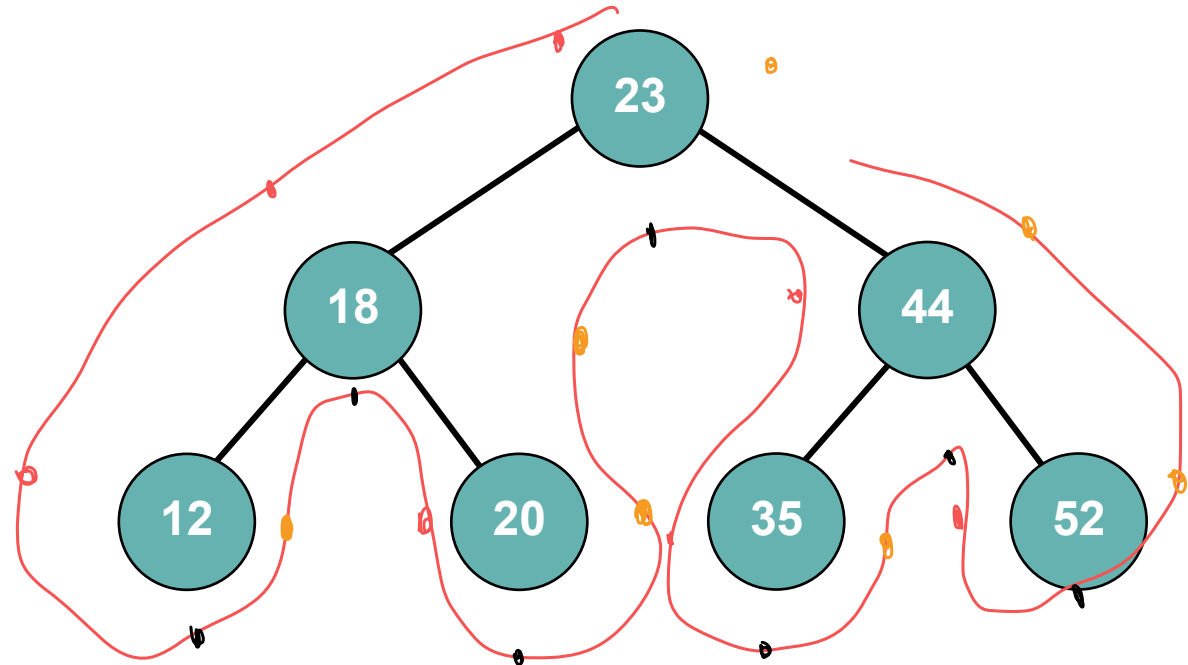
- Traversals
- Search
- Insertion
- Delete

* ໄດ້ວ່າກໍ່ມີໄດ້

ສູ່ດ້ານນີ້ໄດ້ຍັງມີ

Binary Search Tree

TRAVERSALS



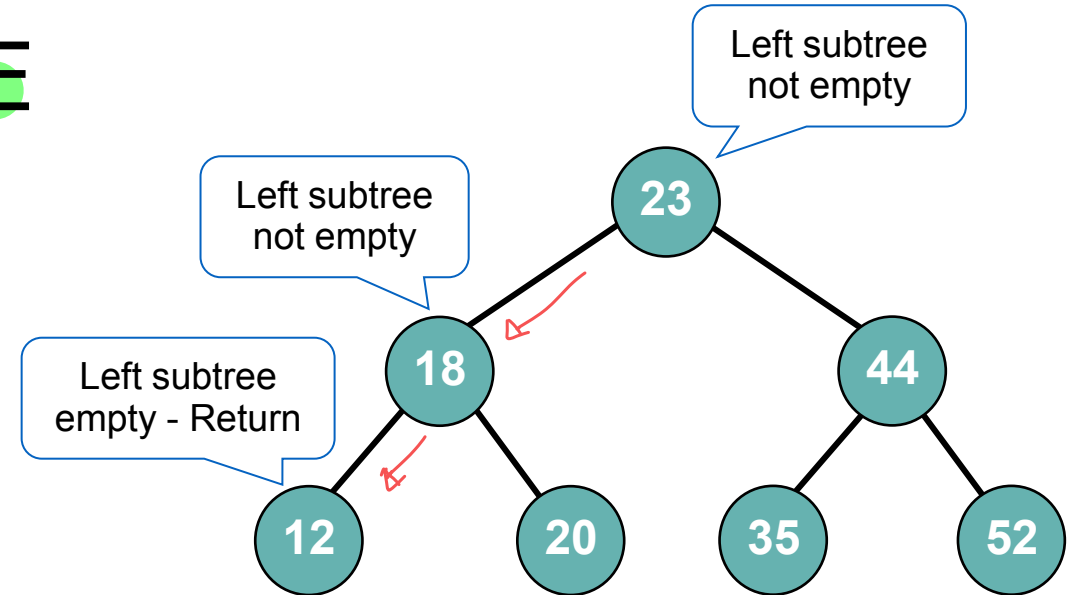
- Preorder Traversal : 23 18 12 20 44 35 52
- Inorder Traversal : 12 18 20 23 35 44 52 → sequence list (list → array)
- Postorder Traversal : 12 20 18 35 52 44 23

SEARCH

- การค้นหาโหนดที่มีค่าน้อยสุด
- การค้นหาโหนดที่มีค่ามากที่สุด
- การค้นหาโหนดที่ต้องการ

FIND THE SMALLEST NODE

- คิดง่าย ๆ -> โหนดที่มีค่าน้อยสุด ก็คือ โหนดที่อยู่ซ้ายสุดนั่นเอง ซึ่งก็คือ 12
- ต้องไปทางซ้ายเรื่อยๆ

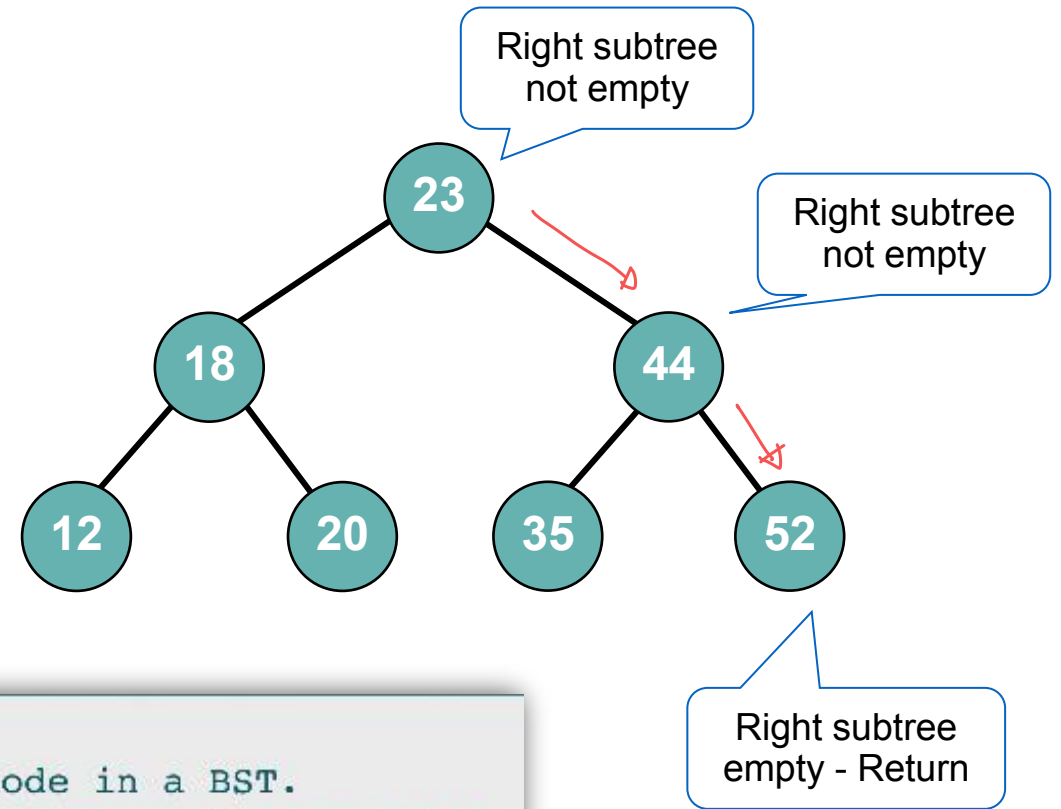


recursive

```
Algorithm findSmallestBST (root)
This algorithm finds the smallest node in a BST.
Pre    root is a pointer to a nonempty BST or subtree
Return address of smallest node
1 if (left subtree empty)
  1 return (root)
2 end if
3 return findSmallestBST (left subtree)
end findSmallestBST
```

FIND THE LARGEST NODE

- คิดง่ายๆ -> โหนดที่มีค่ามากที่สุด ก็คือ โหนดที่อยู่ขวาสุดนั่นเอง ซึ่งก็คือ 52
- ท่องไปทางขวาเรื่อยๆ

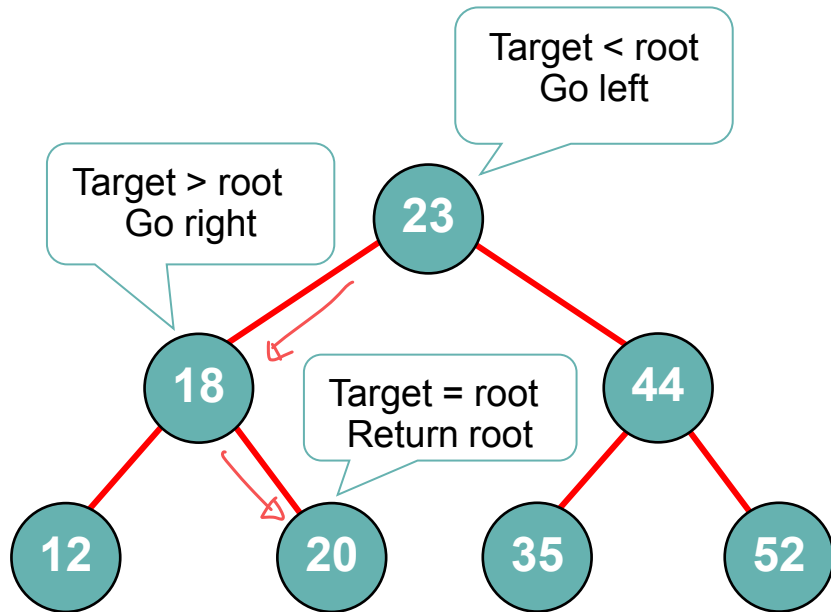


recursive

```
Algorithm findLargestBST (root)
This algorithm finds the largest node in a BST.
Pre    root is a pointer to a nonempty BST or subtree
Return address of largest node returned
1 if (right subtree empty)
1   return (root)
2 end if
3 return findLargestBST (right subtree)
end findLargestBST
```

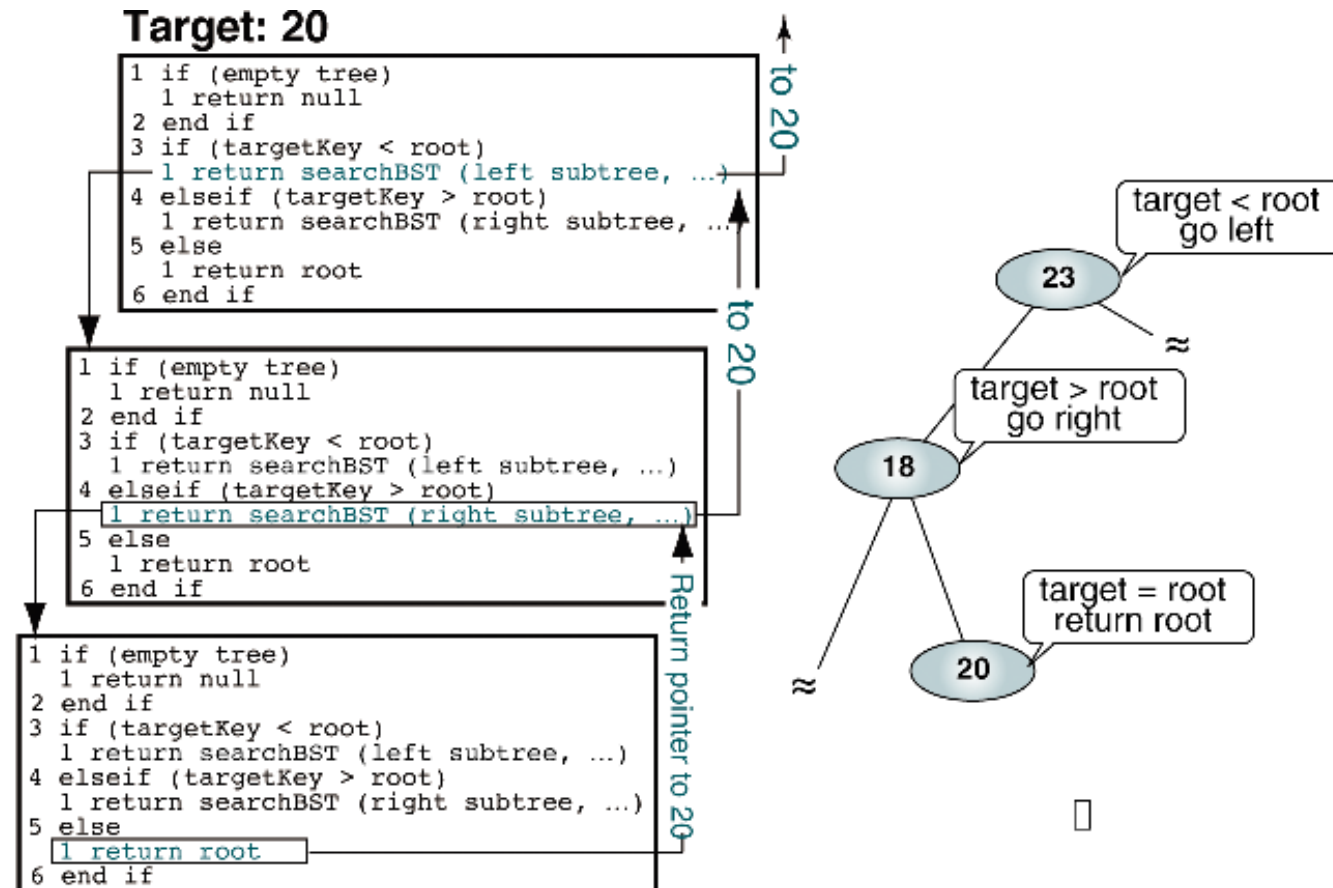
SEARCHING A GIVEN NODE IN A BST

recursive



```
Algorithm searchBST (root, targetKey)
Search a binary search tree for a given value.
  Pre    root is the root to a binary tree or subtree
         targetKey is the key value requested
  Return the node address if the value is found
         null if the node is not in the tree
1  if (empty tree)
    Not found
    1  return null
2  end if
3  if (targetKey < root)
    1  return searchBST (left subtree, targetKey)
4  else if (targetKey > root)
    1  return searchBST (right subtree, targetKey)
5  else
    Found target key
    1  return root
6  end if
end searchBST
```

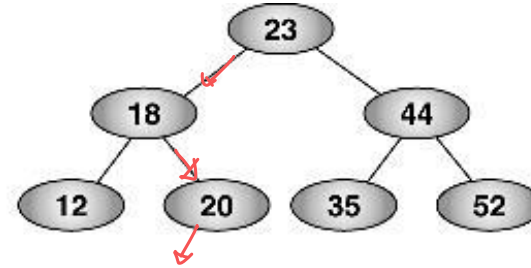
SEARCHING A GIVEN NODE IN A BST



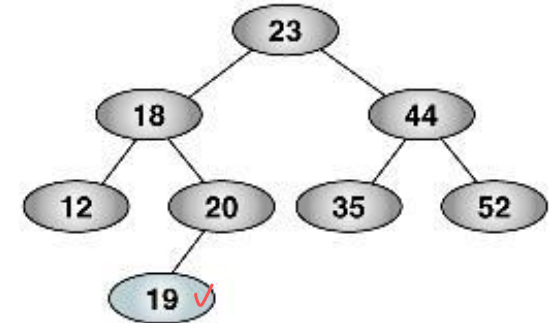
INSERTION

- แทรกข้อมูลเข้าไปใน BST โดย
 - ตามกิ่ง (Branch) ไปเรื่อยๆ จนเจอ subtree ที่ว่าง
 - แล้วแทรกโหนดข้อมูลใหม่เข้าไป (แทรกจาก leaf node)

Ex. Insert 19

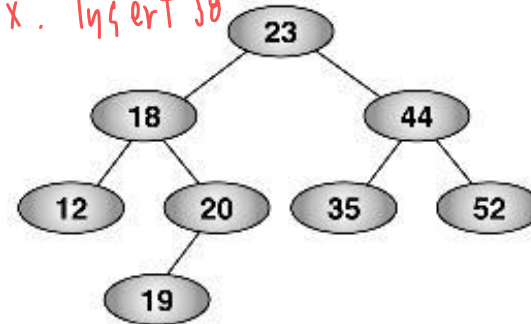


(a) Before inserting 19

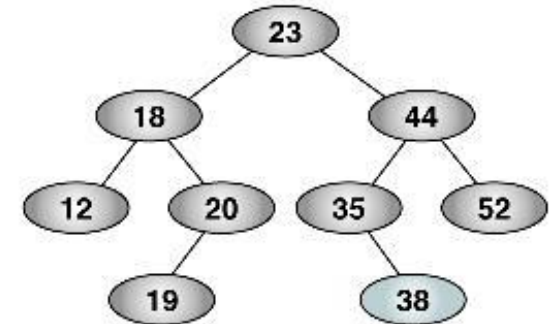


(b) After inserting 19

Ex. Insert 38



(c) Before inserting 38



(d) After inserting 38

INSERTION (CONT.)

Algorithm addBST (root, newNode)

Insert node containing new data into BST using recursion.

Pre root is address of current node in a BST

 newNode is address of node containing data

Post newNode inserted into the tree

Return address of potential new tree root

1 if (empty tree)

1 set root to newNode

2 return newNode

2 end if

Locate null subtree for insertion

3 if (newNode < root)

1 return addBST (left subtree, newNode)

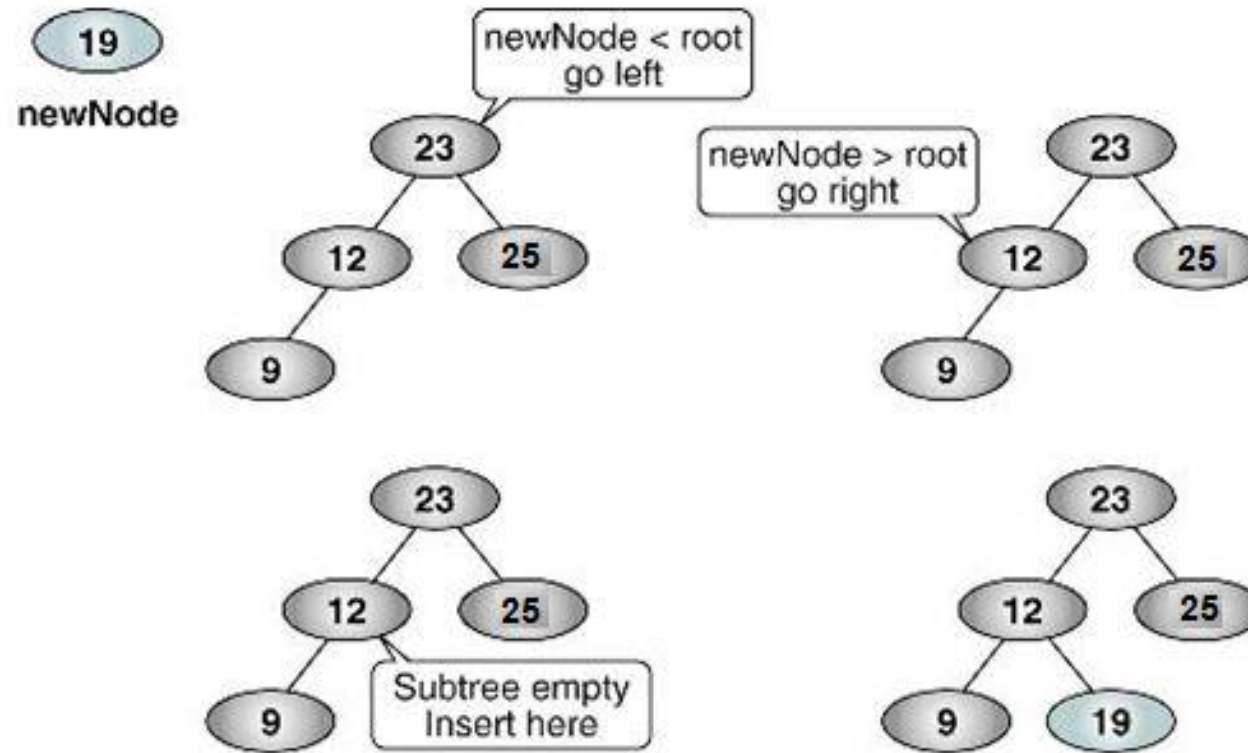
4 else

1 return addBST (right subtree, newNode)

5 end if

end addBST

INSERTION (CONT.)



DELETION

- ❖ จะลบข้อมูลจาก BST ต้องทำการค้นหาโหนดที่ต้องการจะลบให้ได้ก่อน ซึ่งโหนดเหล่านั้นแบ่งลักษณะได้เป็น 4 กรณี
 - โหนดที่จะลบเป็น leaf node (ไม่มีลูก) -> ลบได้เลย
 - โหนดที่จะลบ มีเพียง Right subtree -> เปลี่ยนให้พ่อของโหนดนั้นชี้ไปที่ Right subtree แทน แล้วลบโหนดนั้น
 - โหนดที่จะลบ มีเพียง Left subtree -> เปลี่ยนให้พ่อของโหนดนั้นชี้ไปที่ Left subtree แทน แล้วลบโหนดนั้น
 - โหนดที่จะลบ มีทั้ง Right และ Left subtree -> คัดลอกข้อมูลมากที่สุดของ Left subtree มาที่โหนดที่จะลบ แล้วลบโหนด แล้วลบโหนดมากที่สุดของ Left subtree แทน

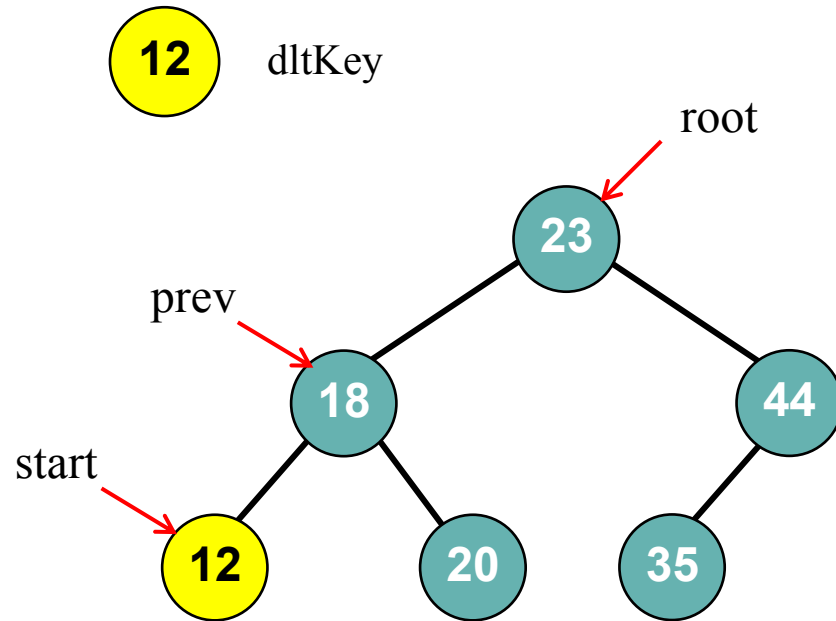

```

Algorithm deleteBST (root, dltKey)
This algorithm deletes a node from a BST.
  Pre    root is reference to node to be deleted
         dltKey is key of node to be deleted
  Post   node deleted
         if dltKey not found, root unchanged
  Return true if node deleted, false if not found
1 if (empty tree)
1   return false
2 end if
3 if (dltKey < root)
1   return deleteBST (left subtree, dltKey)
4 else if (dltKey > root)
1   return deleteBST (right subtree, dltKey)
5 else
  Delete node found--test for leaf node
1  If (no left subtree)
1  make right subtree the root
2  return true
2  else if (no right subtree)
1  make left subtree the root
2  return true
3  else
    Node to be deleted not a leaf. Find largest node on
    left subtree.
1  save root in deleteNode
2  set largest to largestBST (left subtree)
3  move data in largest to deleteNode
4  return deleteBST (left subtree of deleteNode,
                    key of largest
4  end if
6 end if
end deleteBST

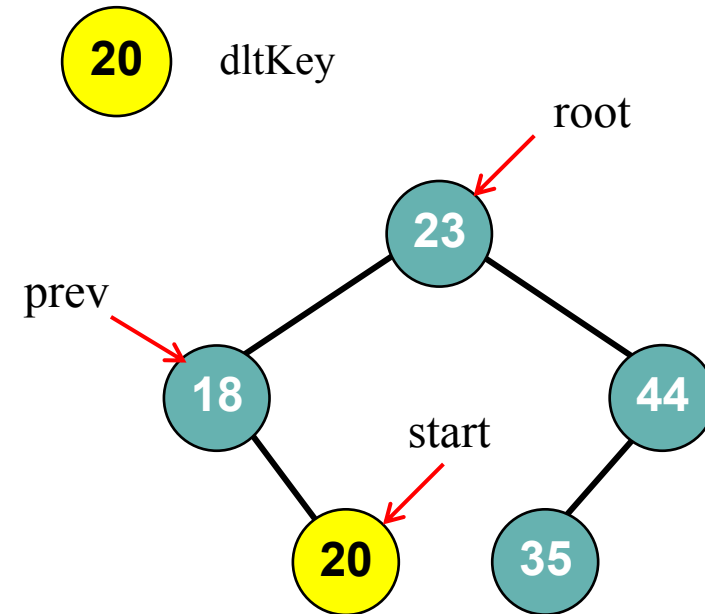
```

DELETION (CONT.)

CASE 1 : THE NODE HAS NO CHILDREN



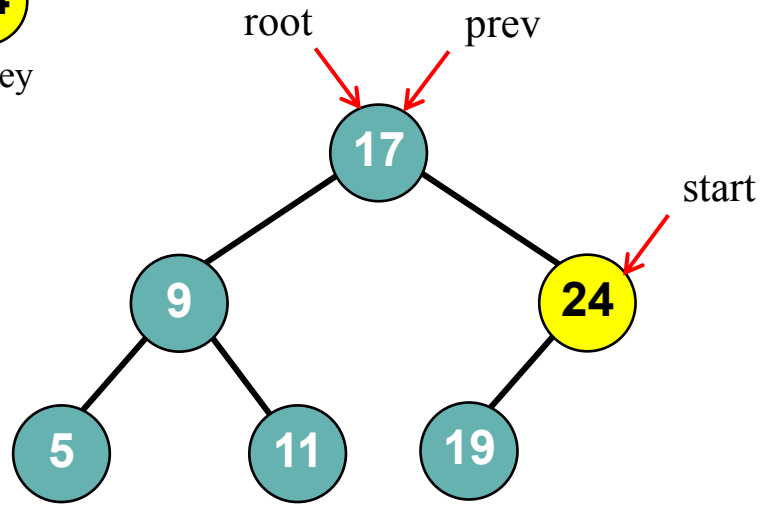
`prev.left = NULL;`



`prev.right = NULL;`

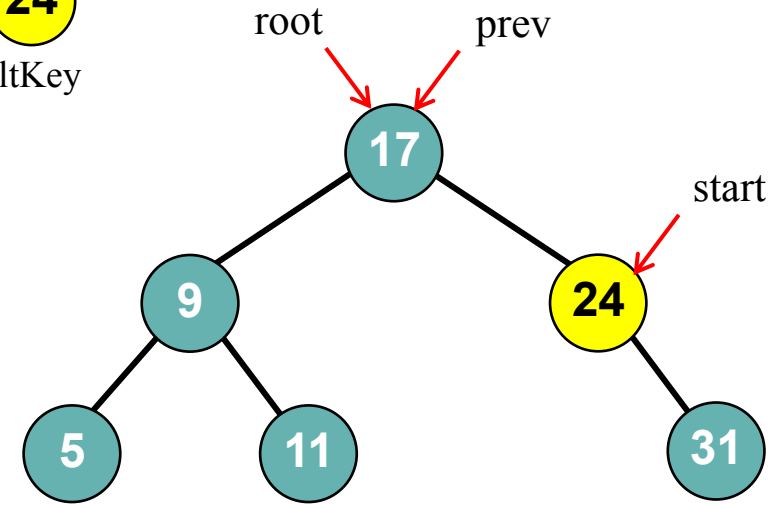
24

dltKey



24

dltKey

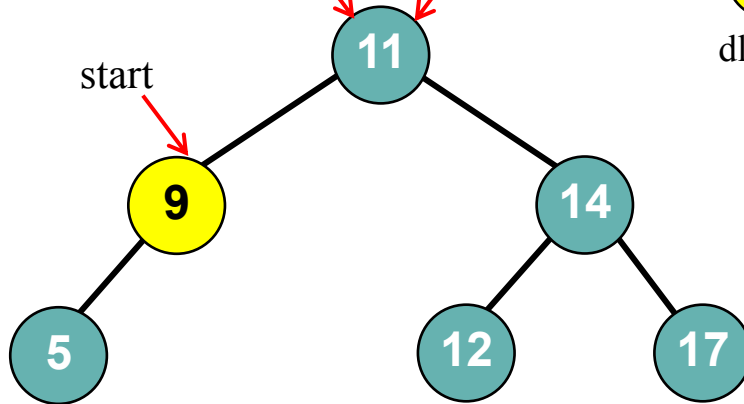


root prev

9

dltKey

start



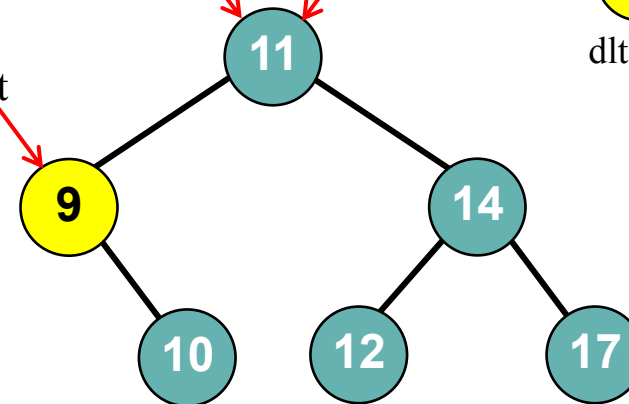
prev.left = start.left

root prev

9

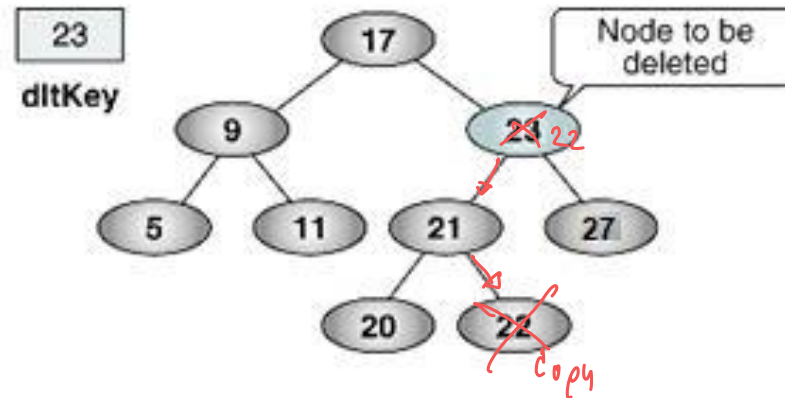
dltKey

start

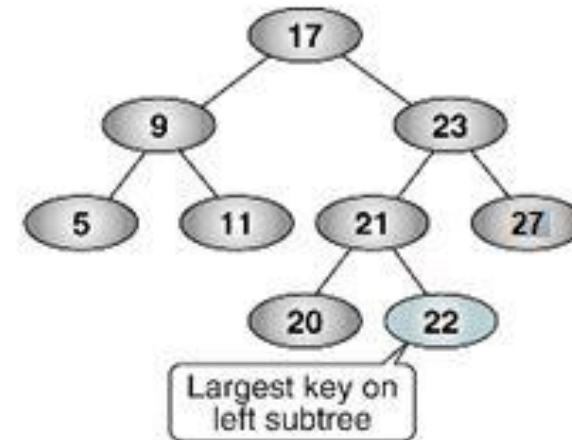


prev.left = start.right

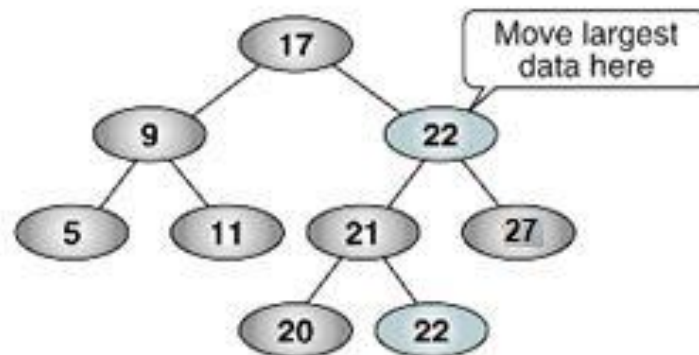
CASE 4 : THE NODE HAVE 2 SUBTREES



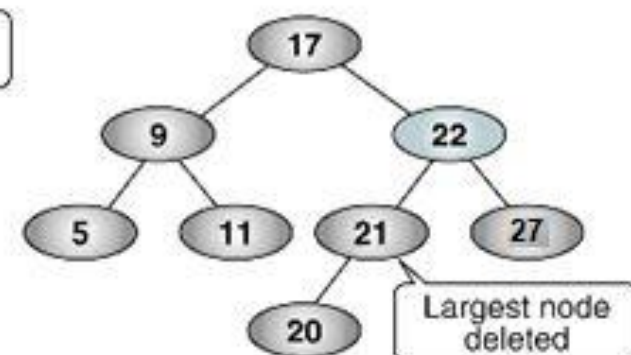
(a) Find dltKey



(b) Find largest



(c) Move largest data



(d) Delete largest node

EXERCISE 1-3

- ❖ ให้วาดรูป BST ตามลำดับข้อมูลที่เข้ามาดังนี้
 - 70, 80, 60, 90, 20, 40, 25, 82, 10, 33
- ❖ ให้วาดรูป BST ตามลำดับข้อมูลที่เข้ามาดังนี้
 - 10, 20, 25, 33, 40, 60, 70, 80, 82, 90
- ❖ ให้ลบโหนดที่มีข้อมูล 70 ออกจาก BST แล้ววาดรูป BST ใหม่

